Component Document Model v0.1 Amendment

1. Top-Level Object Syntax

Every textual document begins with:

```
component:<kind>
```

Version 0.1 defines four object kinds:

```
component:component
component:board
component:operation
component:schema
```

The identifier before `:` names the language family.

The identifier after `:` names the object kind.

---

2. Circuit Description

```
component:component CounterDemo {
    use standard.lib;

    device Clock is OSCILLATOR;
    device Counter is 74HC161;
    device R[4] is RESISTOR;
    device LED[4] is LED;

    Counter[Q0, Q1, Q2, Q3]
        as Counter.Q[0..3];

    connect Clock.OUT -> Counter.CLK;
    connect Counter.Q[0..3] -> R[0..3].1;
    connect R[0..3].2 -> LED[0..3].A;
    connect LED[0..3].K -> GND;

    probe Counter.Q[0..3];
}
```

`component:component` owns:

- Device instances
- logical and physical references
- collections
- connectivity
- probes associated with the machine
- machine-level virtual Devices
- topology

It does not own visual layout.

---

3. Board Description

```
component:board MainBoard {
    use CounterDemo;

    place Clock at 80, 180;
    place Counter at 320, 180;

    place R[0..3] at 560, 100
        direction vertical
        gap 24;

    place LED[0..3] at 760, 100
        direction vertical
        gap 24;

    route Clock.OUT -> Counter.CLK
        style orthogonal;

    route Counter.Q[0..3] -> R[0..3].1
        style bus;

    route R[0..3].2 -> LED[0..3].A
        style parallel;

    show port_name;
    show pin_number;
    show signal_value;

    hide power_wire;

    panel component at left;
    panel board at center;
```

```
    panel terminal at bottom;
}
```

`component:board` owns:

- Device placement
- size and orientation
- routes and wire presentation
- grouping for visual presentation
- labels
- visibility
- zoom and viewport defaults
- Board widgets
- panel arrangement
- display modes
- timeline and waveform placement

It must not redefine circuit topology.

---

4. Operation Description

```
component:operation NoiseTest {
    inject noise
        into Clock.OUT
        during tick 50..100;

    probe Clock.OUT;
    probe Counter.Q[0..3];

    run during tick 0..200;
}
```

The canonical machine form may still be JSON:

```
{
  "component:operation": {
    "name": "NoiseTest",
    "inject": {
      "noise": {
        "into": "Clock.OUT",
        "during": {
          "tick": "50..100"
        }
      }
```

```
    }
  }
}
```

`component:operation` owns:

- requests
- queries
- run control
- simulation steps
- injection
- inspection
- validation
- export
- probe requests

---

5. Schema Description

```
component:schema DeviceSchema {
    object device;

    require name;
    require type;
    require ports;

    optional package;
    optional timing;
    optional visual;
}
```

`component:schema` defines contracts for:

- Components
- Boards
- Operations
- Device Libraries
- Results
- external packages

The schema grammar remains provisional in v0.1.

---

6. Relationship Between Component and Board

```
component:component
        │
        │ describes the machine
        ▼
Components Platform
        │
        │ resolves and executes
        ▼
Resolved Component Model
        │
        │ is presented by
        ▼
component:board
```

The Board references the Component:

```
component:board MainBoard {
    use CounterDemo;
}
```

The Board must not duplicate:

- Device type
- port definition
- physical pin mapping
- connection semantics
- simulation behavior

The Board may describe how an existing connection is shown:

```
route Counter.Q[0..3] -> R[0..3].1
    style bus;
```

But it may not create a new circuit connection unless that interaction is translated into an Operation that modifies the Component.

---

7. File Organization

Recommended package:

```
CounterDemo/
├── main.component
├── main.board
├── operations/
│   └── noise_test.operation
└── component.json
```

`main.component`

```
component:component CounterDemo {
    ...
}
```

`main.board`

```
component:board MainBoard {
    use CounterDemo;
    ...
}
```

`noise_test.operation`

```
component:operation NoiseTest {
    ...
}
```

---

8. Multiple Objects in One File

Version 0.1 may permit one object per file for simplicity.

A future version may permit:

```
component:component CounterDemo {
    ...
}

component:board MainBoard {
    use CounterDemo;
    ...
}
```

```
component:operation DefaultRun {
    run during tick 0..200;
}
```

The parser should therefore treat each `component:<kind>` block as an independent top-level object.

---

9. Parser Rule

The first production becomes:

```
document
    := object*

object
    := "component" ":" object_kind identifier? block

object_kind
    := "component"
     | "board"
     | "operation"
     | "schema"
```

Examples:

```
component:component CounterDemo { }
component:board MainBoard { }
component:operation RunTest { }
component:schema DeviceSchema { }
```

---

10. JSON Envelope

Canonical JSON should mirror the textual notation:

```
{
  "component:component": {
    "name": "CounterDemo",
    "devices": [],
    "connections": []
```

```
  }
}
```

```
{
  "component:board": {
    "name": "MainBoard",
    "uses": "CounterDemo",
    "placements": [],
    "routes": []
  }
}
```

```
{
  "component:operation": {
    "name": "NoiseTest",
    "operations": []
  }
}
```

```
{
  "component:schema": {
    "name": "DeviceSchema",
    "object": "device"
  }
}
```

This keeps text and JSON structurally parallel.

---

11. Ownership Rule

```
component:component
    owns circuit meaning

component:board
    owns visual presentation

component:operation
    owns requests and experiments

component:schema
    owns contracts
```

No object kind may silently redefine another object's responsibility.

---

12. Frozen v0.1 Decision

The following top-level syntax is frozen:

```
component:component { }
component:board { }
component:operation { }
component:schema { }
```

Named forms are also supported:

```
component:component CounterDemo { }
component:board MainBoard { }
component:operation NoiseTest { }
component:schema DeviceSchema { }
```

The colon `:` is reserved at the top level for identifying the Component object kind.